CS3423 Mini Assignment 6: Q5

Q5: mem2reg Pass Analysis

Setup

I generated `before.ll` and `after.ll` for each of the 5 programs using the commands (given in README):

```
clang++ -std=c++17 -S -emit-llvm -O0 -Xclang -disable-O0-optnone "cpp Codes/N.cpp" -o IR/N_before.ll
opt -passes=mem2reg -S IR/N_before.ll -o IR/N_after.ll
opt -passes=verify -disable-output IR/N_after.ll
```

The flag `-disable-O0-optnone` is needed because clang normally adds the `optnone` attribute to functions compiled with `-O0`. This attribute tells LLVM to skip most optimization passes for that function, including mem2reg.

(a) What analysis information does mem2reg depend on?

The mem2reg pass uses information about the control flow graph and the definitions and uses of variables to convert eligible variables stored in memory into SSA form.

Analysis	What it provides / how it is used
DominatorTreeAnalysis	Provides the dominator tree, used to compute where PHI nodes are needed and to rename values during SSA construction.
AssumptionAnalysis	Provides `llvm.assume` information, used so that assumes on a promoted value are handled correctly.

- **Iterated dominance frontier:** mem2reg computes this from the dominator tree to decide where PHI nodes must be placed for each promoted variable. It is not a separate analysis pass.
- **Promotability check (definitions and uses):** an alloca is promoted only if it has a single-value type (not an array or struct) and every use is a non-volatile load or store that uses the alloca as its pointer operand. If its address is passed to a call, stored somewhere, used in a GEP, or fed into a PHI/select, it is not promoted.

Using this information, mem2reg inserts PHI nodes where required and renames variables so that each SSA value has a single definition.

(b) What variables are converted to SSA form?

Program	Function	Allocas before	Allocas after	PHIs before	PHIs after
1	basicPointer	3	0	0	0
2	conditionalPointer	4	2	0	1
3	pointerEscape + modify	4	1	0	0
4	arrayPointer	5	1	1	3
5	heapPointer	4	0	0	2

Program 1: basicPointer

The variables `input`, `value` and `ptr` are promoted to SSA form. Their corresponding alloca, load and store instructions are eliminated.

Even though the address of `value` is initially stored in `ptr`, both variables can be promoted. Once `ptr` is promoted, its uses become direct uses of the address of `value`. This allows `value` to be promoted as well. The resulting IR uses SSA values directly instead of accessing these variables through memory.

Program 2: conditionalPointer

The variables `condition` and `ptr` are promoted to SSA form. Their corresponding stack allocations and associated loads and stores are removed.

A pointer-valued PHI node is introduced at the join block. It selects between the address of `first` and the address of `second`, depending on which branch was executed. The selected pointer is then used to access and update the corresponding integer.

Program 3: pointerEscape

In `pointerEscape()`, the variables `n` and `ptr` are promoted to SSA form. In `modify()`, the stack allocation used to store the parameter `p` is also eliminated because the parameter is already available as an SSA value.

The corresponding alloca, load and store instructions for these promoted variables are removed. However, the actual variable `value` remains memory-backed because its address is passed to `modify()`.

Program 4: arrayPointer

The scalar variables `n`, `ptr`, `sum` and `i` are promoted to SSA form. Their corresponding stack allocations and associated loads and stores are removed.

PHI nodes are introduced for the loop-carried variables `sum` and `i`, since their values are updated during each loop iteration. The IR also contains a PHI node associated with the short-circuit evaluation of the `&&` condition.

Program 5: heapPointer

The local stack variables `n`, `buffer`, `sum` and `i` are promoted to SSA form. Their corresponding stack allocations and associated loads and stores are eliminated.

Two PHI nodes are introduced at the loop header for `sum` and `i`, since their values are carried from one iteration to the next. The pointer variable `buffer` is also promoted, but the dynamically allocated array it points to remains in heap memory.

(c) What variables are NOT converted to SSA form and why?

Program 1

No variables remain unpromoted. All three local variables (`input`, `value` and `ptr`) are successfully converted to SSA form.

Program 2

The variables `first` and `second` remain as allocas. Their addresses are used as incoming values of the pointer PHI node, which is a use other than a direct load or store. They therefore fail the promotability check, and the loads and stores through the selected pointer stay in memory.

Program 3

The variable `value` remains as an alloca in memory. Its address is passed to the function `modify()`, which accesses and modifies the value through the pointer. Therefore, mem2reg cannot promote this variable in this example.

Program 4

The array `arr` remains memory-backed. It has an array type rather than a single-value type, and its elements are accessed through GEP instructions, which are not plain loads or stores of the alloca. mem2reg does not promote such allocas (splitting an aggregate into scalars is the job of SROA, not mem2reg).

Program 5

The dynamically allocated array created using `new int[n]` remains in heap memory. mem2reg operates on eligible stack allocations created using `alloca`, not dynamically allocated heap objects.

The local pointer variable `buffer`, which stores the address of this heap allocation, is promoted to SSA form. However, the actual heap array and its load/store operations remain unchanged.

Conclusion

The mem2reg pass converts promotable stack variables into SSA values, removing their alloca, load and store instructions. It uses the dominator tree to place PHI nodes where values from different control-flow paths merge. Allocas whose address escapes (passed to a call, stored, or used in a PHI or GEP), aggregate allocas such as arrays, and heap objects are not promoted and remain memory-backed.